אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

July 26, 2026

Abstract

Semantic caches are usually evaluated on hit rate, latency, and at best false-hit rate. None of these tells whether a hit was correct at the moment it was served. I give that gap a name and a number: *stale-hit-rate*, the fraction of served cache hits whose content had already expired by the time it was served. I extend GPTCache, a frozen and widely used open-source semantic cache, with a per-cluster staleness model fit from observed data. The model is used in two places: as a validity gate on the serve path, and as a soft decay term inside eviction. Across a bracketing experiment, a six-row ablation, three parameter sweeps, and two model-misspecification checks, all run with ten seeds per configuration, the gate cuts stale-hit-rate roughly 19-fold against a frequency-based floor while removing only about 3% of useful hits, and useful-hit-rate stays flat across most of the confidence range I swept. A per-cluster learned decay rate beats a single pooled rate by a wide, significant margin, and keeps doing so even when the workload is easier to predict than the fitter assumes. I also test a harder, adversarial case where the misspecification is genuine. Throughout, I measure the trade-off at the level of individual queries rather than only in hit-rate percentages, and I report which of the two mechanisms, the gate or the eviction term, is actually doing the correctness work.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response whenever an incoming query lands close enough in embedding space to one it has already answered. The promise is simple: skip the call, return the same answer faster and for free. But "the same answer" is a claim about time as well as similarity, and most semantic caches never check the time part.

This matters most in systems that put a language model between a user and an answer about content that changes on its own schedule, such as monitoring how a brand is represented across AI-generated answers (answer engine optimization, or AEO). These systems ask the same style of question over and over, exactly the shape of workload a semantic cache is built to absorb, but the facts behind the questions do not sit still: a product goes out of stock, a price changes, a review's sentiment shifts. A cache that only asks "have I seen a similar question before" and never "is that answer still true" keeps serving a stale answer indefinitely once written, and whatever sits on top of the cache has no way to tell a correct hit from a wrong one, because stale-hit-rate simply does not exist in standard cache evaluation.

Two concrete gaps follow, and they motivate the two components I add. **No staleness concept in the serve path:** a cached answer about a fact that changes on its own schedule

is served indefinitely once written, with "found in cache" silently treated as "correct" and no mechanism to ever reconsider that judgment. **No cost term in eviction:** count-based policies such as LRU and LFU treat two entries with identical access patterns the same way even when one is far more expensive to regenerate, so a policy that ignores regeneration cost can evict a rarely accessed but expensive entry ahead of a cheap one that was merely accessed once more recently.

Industry and research reflect this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock time-to-live and no similarity matching at all. Semantic-caching products use a single fixed global similarity threshold with plain time-to-live or count-based eviction, so neither has any notion of per-topic staleness. The research literature is further ahead, proposing cost-aware and staleness-aware eviction formulas, but none names or measures stale-hit-rate. I compare this work against that literature, not against the weaker product baseline.

This report makes four contributions. First, stale-hit-rate as a named, measured evaluation metric. Second, a per-cluster staleness model, fit from data, that is consumed both as a serving-time validity gate and as an eviction-time decay term. Third, an ablation that isolates which of the two consumers is actually doing the correctness work. Fourth, two misspecification checks that test whether the staleness model's own assumption, that content validity decays exponentially, matters to the result; the stronger of the two is a genuinely adversarial two-rate mixture within each cluster. Section 2 justifies the baseline, Section 3 describes the design, Section 4 covers the workload and harness, and Section 5 reports seven experiments in total: one bracketing run, one six-row ablation, three parameter sweeps, and two misspecification checks.

1.2 Related work

The closest prior art, and my primary eviction comparator, is Biton and Friedman's work on semantic-cache eviction [1]. It proves that optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality; it evaluates these on GPTCache and releases the code, part of why I chose GPT-Cache here. Two findings shape my design directly: LRU is weak on semantic workloads, and frequency-based policies are a much stronger baseline, which is why every comparison in this report treats an LFU-class policy, not LRU, as the floor. FreCoS (**Fre**shness- and **Co**st-aware **S**erving) differs in that their policies combine access-pattern signals only, while FreCoS adds two orthogonal signals, regeneration cost and content validity over time, and evaluates against a correctness metric, stale-hit-rate, that their setting never defines.

The structurally closest published eviction formula is Cortex's LCFU policy [2] (earlier preprint: Asteria), which scores an entry as a product of log-scaled frequency, cost, latency, and a staleness term, divided by size, with a time-to-live purge on top. The difference is narrow: Cortex's staleness term is a static "staticity" score assigned by a language model at write time, while FreCoS learns a decay rate per cluster from observed staleness; Section 5.1 tests whether that learning does measurable work. I did not reimplement Cortex's formula, since it relies on remote tool-call latency metadata with no equivalent here.

Several other lines touch individual pieces of the design without matching the combination. GDSF [3] ages access recency, not content validity, so it has no freshness semantics at all. Category-Aware caching [4] assigns per-category time-to-live values that are load-based rather than learned and defines no stale-hit-rate equivalent; this is also why I cut a third, originally planned component, calibrated per-cluster similarity thresholds, a lossy special case of both this work and per-embedding threshold learning. FreshCache [5] shares FreCoS's lineage of putting staleness in the serve path, but as a probabilistic risk budget per tier rather than a hard gate; I list it as future work. SCALM [6] clusters cache entries similarly to my k-means step, but its significance score is not staleness-aware. MeanCache [7] is cost-motivated but does not target

eviction. W-TinyLFU [8] is admission control and out of scope, an orthogonal axis to both my contributions. I also considered and rejected vCache [9]: its contribution is per-prompt threshold learning with eviction explicitly out of focus, so building this work on top of it would have diluted the comparison rather than sharpened it.

2 Baseline

2.1 Why GPTCache

I chose to extend GPTCache [10] rather than a more actively maintained semantic cache. The case for it rests on architecture and control properties, not on community activity, for four reasons. First, GPTCache's most recent release is v0.1.44 (August 2024); it carries dozens of open issues and pull requests, and the maintainers state outright that support for new integrations has stopped. Ordinarily that would count against a dependency, but here it works in my favor: a frozen interface is a better scientific control than a moving one, and the baseline I measured early in this project is byte-identical to the one I measured at the end. The second reason is architectural fit, verified by reading the vendored source directly rather than trusting the documentation. Every component I touch, the similarity threshold check, the hit and miss decision, the eviction registry, sits behind a registered, named interface, so my extension is purely additive: a new eviction subclass registered through the existing hook, a metadata field added via composition around storage, and a serving-path gate plugged into one shared decision helper; I never edit vendored code. Third, GPTCache is the baseline the closest prior art already uses, which in principle lets me use Biton and Friedman's released policy directly as my primary comparator, though in practice I ended up substituting a documented proxy, for reasons explained in Section 5.2. Fourth, its default embedding backend runs on the CPU deterministically, keeping every benchmark run reproducible without a GPU.

One consequence worth flagging: the course guidelines offer a grade bonus for an upstream contribution adopted by maintainers, but GPTCache's own README states that new contributions are not being merged, so choosing GPTCache for the reproducibility argument above forecloses that bonus, a trade made deliberately.

2.2 GPTCache's behavior and default eviction policy

GPTCache serves a cached response when an incoming query is semantically close enough to one it has already seen, using a single global cosine similarity threshold (0.8 by default) rather than exact-match keys. Its eviction policies are purely count-based: least-recently-used, least-frequently-used, first-in-first-out, or random replacement. None of them reads generation time, access recency, or regeneration cost beyond the single counter each already tracks, and none retains metadata on an evicted entry at all. Creation and last-access timestamps are persisted per entry but never read by any decision logic, columns with no downstream consumer, which is precisely the gap my extension repurposes. There is no admission control; every miss gets written back unconditionally. No cost accounting, false-hit tracking, or staleness tracking exists anywhere in the built-in reporting layer, which counts only per-stage operation timers.

3 Design

3.1 Contribution

The contribution boils down to one learned artifact with two consumers. FreCoS fits a per-cluster staleness model and uses it at both ends of the cache's lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The

metric that makes both uses visible is stale-hit-rate: the fraction of served hits whose content was already invalid by the time it was served.

Cluster assignment comes from a fixed, seeded k-means clustering over cached-query embeddings, run once at cache-build time with no online re-clustering, a deliberate reproducibility choice. I compute these embeddings because the staleness model needs cluster identity; Section 4.2 explains why the benchmark harness used to evaluate the model does not also use them for cache lookup. Once assigned, both the gate and the eviction policy consume the same cluster identity and neither recomputes it independently, since a gate and an eviction policy that quietly disagree about an entry's cluster would be one of the more likely integration bugs in a system shaped like this one; I checked for it directly in the ablation (Section 5.2).

For each cluster, a decay rate λ_c is fit from observed staleness in a held-out calibration split, assuming validity decays exponentially: $\Pr[\text{still valid after } \Delta t] = e^{-\lambda_c \Delta t}$. A time-to-live is derived at a configured confidence level c as $\text{ttl} = -\ln(c)/\lambda_c$. Three interchangeable modes produce this table with the same output shape: *learned* fits each cluster's rate independently by maximum likelihood, falling back to a rate pooled across all clusters below thirty observations; *global* pools every cluster into one shared rate regardless of sample size; *oracle* takes the rate from the workload generator's ground truth, an upper bound never available in a real deployment. Table 1 lists every tunable parameter, its default, and where its effect is measured.

The validity gate is the first consumer. On a candidate hit, if the entry's age exceeds its cluster's time-to-live, it gets treated as a miss, the answer is regenerated, and the event counts as a stale hit prevented rather than served. This is a single comparison against creation time; the gate never reads last-access time, because staleness is a property of when an answer was generated, not how recently someone asked for it, and a popular but outdated entry should not be treated as fresh just because it is popular.

The eviction value function is the second consumer:

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen\_cost}) \cdot e^{-\lambda_c \cdot \text{age}} \,/\, \text{size\_bytes} \tag{1}$$

where age is measured from creation time, never last access; $\kappa = 1000$ log-compresses regeneration cost (typically a few tenths of a US cent) so its heavy-tailed distribution cannot dominate the other terms; and the victim is the entry with the lowest value, ties broken by oldest creation time then lowest entry identifier.

One detail comes from a real design mistake I ran into and had to fix. My original frequency term was $\log(1 + \text{freq})$, exactly zero for a never-accessed entry. Because the value function is a product of four terms, a zero frequency term zeroes out the entire score regardless of cost, freshness, or size, so a freshly written entry would always get evicted next, which is obviously wrong. A property test built around exactly this claim, described in Appendix A, failed against the literal formula and caught the bug; I fixed it by changing the term to $\log(2 + \text{freq})$ in Equation 1, which keeps the same concave shape but guarantees a strictly positive score at zero frequency. This fix is applied throughout every experiment in this report.

Because the gate already refreshes anything past its time-to-live, the decay term inside the value function is not doing staleness prevention on its own. Instead it acts as a soft prior that favors entries with more remaining useful life among the candidates the gate has already judged fresh. That is the intended division of labor, and the ablation in Section 5.2 tests it directly.

Eviction remains entry-count budgeted, matching GPTCache's convention. Entry size in bytes is recorded and used only inside the value function's denominator; I did not build a separate byte-budget eviction loop.

An earlier design included a third component, calibrated per-cluster similarity thresholds, which I cut before implementation began. It turns out to be a lossy special case of two things already in the literature, vCache's per-embedding threshold learning and Category-Aware caching's per-category treatment, so building a third, weaker version would have cost more than it contributed. It remains future work.

Table 1: Every tunable parameter in the extension, its default, and where its effect on stale-hit-rate or hit rate is measured in Section 5. TTL confidence $c = 0.95$ is the default `Config` ships with, set from the sweep result in Section 5.3: useful-hit-rate is flat from 0.80 through 0.95, so 0.95 buys a further stale-hit-rate reduction over 0.90 at no measured cost. The bracketing and ablation experiments in Sections 5.1–5.2 were run at $c = 0.90$, since they predate this sweep; that value is called out explicitly wherever those experiments' results appear.

Parameter	Default	Swept	Measured in
lambda source (mode)	learned	global, learned, oracle	5.1
TTL confidence c	0.95	0.80, 0.90, 0.95, 0.99	5.3
cluster count K	10	5, 10, 20, 50	5.3
cache size (entries)	1650 (see note)	5%–80% of working set	5.3
κ (cost log-compression)	1000	not swept	—
calibration fallback threshold	30 obs.	not swept	—
eviction size term	on	on / off	5.2

Cache size is 1650 entries in the bracketing and ablation experiments; the cache-size sweep itself spans 5%–80% of the working set and identifies 1980 entries as the knee (Section 5.3), which anchors the TTL-confidence and cluster-count sweeps. κ and the thirty-observation fallback threshold were fixed at values that seemed reasonable at design time; neither was independently swept.

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator I built for this project. Before other implementation work began, I timeboxed a feasibility spike into a second workload, derived from Wikipedia revision history with staleness ground truth from edit timestamps: it sampled thirty question-answer pairs from SQuAD 2.0 [11] and compared an automatic sentence-change rule against twenty hand-labeled revision pairs. The rule agreed with my labels only 40% of the time, well short of the 80% go/no-go bar set in advance, and a reliable rule was estimated to need eighteen to twenty-three more hours than I had. I proceeded with W1 alone; the full spike is in `docs/w2-feasibility.md`, and Section 6 discusses what this limits.

W1 generates queries across several streams inside each topic cluster: canonical queries that establish a ground-truth answer, near-duplicate paraphrases that share that answer's identity, novel long-tail queries that never repeat, and time-shifted repeats of an existing answer. Every cluster draws its own random half-life and regeneration-cost distribution. The generator guarantees that for every repeating answer identity, at least one later occurrence lands past that answer's true expiry, since otherwise the validity gate would have nothing to reject. Each trace is split by arrival time, with the first 30% used for calibration and the remaining 70% for evaluation, so no parameter is ever fit on data it is later scored against.

The default generator draws half-life from an exponential distribution, which matches the staleness fitter's own assumption exactly. Section 5.1.1 reruns the bracketing experiment twice with that assumption deliberately violated: once with half-life drawn from a Weibull distribution instead (same mean, different shape), and once, the stronger of the two checks, with half-life drawn from a mixture of two exponentials within each cluster. The Weibull attempt turned out to be the weaker, discarded direction, for reasons given there.

The generator's two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and observed update-frequency

in a public corpus. Neither turned out to be reachable from my development environment, so I parameterized both from commonly cited public figures instead. That makes them a plausibility check rather than a real external fit, and is a second limit on external validity.

4.2 Simulation model

The benchmark harness every experiment runs through is a pure trace replayer, so no language model is ever actually called. On a miss, the response content is whatever answer identity the trace recorded, the cost charged is the trace's recorded regeneration cost, and miss latency is simulated from a seeded log-normal distribution scaled by entry size, deterministic per query. That distribution is a documented placeholder, not something fit to a real inference trace. Hit latency and the extension's own overhead (cluster lookup plus the gate check) are measured for real with wall-clock timing: median overhead is 0.57 microseconds per decision at the LFU floor and 0.69 microseconds at the full stack (Table 6), which is what the gate and cluster lookup add on top of GPTCache's own decision path. Reported latency in every table is therefore a hybrid of real cache-side measurement and simulated backend cost. That is the right trade for keeping every run free and deterministic, but it is a limitation worth stating plainly.

A further limitation shapes every absolute number below: the harness's lookup index is exact-text-match. I compute embeddings for clustering (Section 3.1) but not for retrieval, so no semantic index is wired in yet, and paraphrase queries, one of W1's four stream types, never register as hits, only literal repeats do. Hit counts across every experiment sit in the low hundreds out of thousands of scored queries (exact per-seed counts appear beside every rate in Section 5), which widens every confidence interval, and it also means false-hit-rate is exactly zero in every run of every experiment in this report, since exact matching always retrieves the entry actually written for that string. That metric carries no signal under this harness, which is why it does not appear as a column in the results tables below; a real semantic index would be the right place to measure it.

Every relative comparison in this report relies on one invariance: this limitation does not favor one policy or lambda source over another, and that holds within each comparison because the index is held fixed across every condition inside it. It would not necessarily survive a swap to a semantic index, though, since a paraphrase match would then inherit an older entry's age rather than a freshly-written one's, changing the age distribution the gate sees, and I have not verified that the gate's or eviction policy's relative ranking would survive that change.

The cache starts empty in every run. The first 10% of the evaluation split warms the cache and is excluded from every metric, so only the remaining 90% is actually scored.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry's true expiry. False-hit-rate is the fraction where the served answer's identity does not match the query's true identity. Useful-hit-rate is the fraction of *all scored queries*, not just hits, that were served a correct, non-stale answer. Cost saved sums regeneration cost over non-stale hits only, since a stale hit served a wrong answer and saved nothing. Every configuration ran ten independent seeds, each with its own freshly generated trace, since the pipeline has no randomness of its own once a trace is fixed. Results are reported as medians with 95% confidence intervals from a percentile bootstrap over the ten per-seed values (10,000 resamples, fixed resampling seed). With only ten seeds a bootstrap CI is coarser than it looks, so I also report raw hit and stale-hit counts alongside every rate. Group comparisons use the Mann-Whitney U test with the rank-biserial correlation as an effect size, since at $n = 10$ the smallest achievable two-sided p-value is about 0.0002, and p alone cannot distinguish a large effect from a small one at that floor. I ran roughly fifteen such tests without correcting for multiple comparisons, so borderline p-values should be read with that in mind. A non-significant result is reported here as an absence of

detected difference at $n = 10$, never as equivalence; where I claim two conditions behave alike, I back that up with matching raw counts, not just a large p-value.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling

This experiment tests the core claim behind fitting a staleness rate per cluster at all: does it do anything relative to two obvious alternatives, a rate pooled across every cluster and the true rate a real system could never observe. I ran three lambda sources, global, learned, and oracle, for 10 seeds each on the same 12,000-query W1 configuration, with the gate enabled at TTL confidence $c = 0.90$ and FreCoS eviction active throughout. (This predates the sweep in Section 5.3 that sets the shipped default to $c = 0.95$; the comparison across lambda sources does not depend on which confidence level is used.)

Table 2: Per-cluster fitting cuts stale-hit-rate 4x relative to a pooled rate and is indistinguishable from an oracle rate the system could never actually observe. Median hit count per run was 130–143 (Table 3).

Lambda source	Median stale-hit-rate	95% CI	Median cost saved (USD)
Global	0.144	(0.085, 0.214)	0.448
Learned	0.035	(0.024, 0.060)	0.449
Oracle	0.035	(0.028, 0.064)	0.449

Table 3: Raw counts behind Table 2: median served hits and stale hits served, out of a median 7560 scored queries per run.

Lambda source	Median n_hits	Median n_stale_hits_served
Global	143	21
Learned	130	4.5
Oracle	130.5	4.5

Learned separates cleanly from global: median stale-hit-rate is 0.035 under learned against 0.144 under global, the confidence intervals do not overlap, and the effect is large (Mann-Whitney $U = 4.0$, $p \approx 0.0005$, rank-biserial $r = 0.92$). Pooling every cluster's observations into a single rate is significantly worse than fitting each cluster's own.

Learned and oracle, on the other hand, are statistically indistinguishable ($U = 49.0$, $p \approx 0.94$, $r = 0.02$): learned's median stale-hit-rate equals oracle's exactly, and eight of ten seed pairs are byte-identical in raw stale-hit count. This is close to guaranteed by construction rather than a surprising result: W1 draws each cluster's true half-life from an exponential distribution, and the staleness fitter assumes exactly that distribution when it fits λ_c, so with 550–650 calibration observations per cluster here, well above the 30-observation fallback, maximum-likelihood estimation recovers the true rate almost exactly. The defensible claim from this run alone is fairly narrow: if content half-lives vary by topic and are close to exponential, per-cluster fitting from a few hundred observations recovers them and a pooled rate does not. It is not yet evidence that learned estimation holds up when its own model is wrong, which is exactly what Section 5.1.1 tests next.

A first follow-up asked whether learned and oracle still separate at a substantially smaller calibration sample, roughly sevenfold sparser (30–70 observations per cluster instead of 550–650). They do not ($p \approx 0.96$), while learned versus global remains significant ($p \approx 0.0015$). This is a confound rather than a property of the fit itself: shrinking the trace shrinks the

calibration sample and the evaluation-split hit count together, and it is the eval-side hit count, not calibration sample size alone, that decides whether estimation noise shows up in a measured stale-hit-rate. Figure 1 shows both runs on one axis.

5.1.1 Misspecification: what happens when the exponential assumption is wrong

My first attempt at this check drew W1's true half-life from a Weibull distribution instead of an exponential one, keeping the same mean per cluster but using shape parameter 2 instead of the exponential's implicit 1. This turned out to be underpowered by construction, not just by accident: a Weibull with shape 2 has roughly half the coefficient of variation of the exponential it replaces, so the change made the workload's staleness *easier* to predict, not harder. Every arm's stale-hit-rate improved (global went from 0.144 to 0.065, learned and oracle both dropped to 0.000), and learned and oracle floored at exactly zero instead of visibly separating, a floor effect from low hit counts, not evidence that either estimator handled misspecification well. I include this attempt because it taught me which axis to vary next, not because the result itself says much on its own.

A genuinely adversarial case needs the exponential's own maximum-likelihood fit to be systematically biased, not just non-exponential with a smaller spread. So I reran the bracket with each cluster's true half-life drawn from a 50/50 mixture of two exponentials, one at a fifth of the cluster's mean scale and one at 1.8 times it, keeping the same overall mean as before. A single fitted rate cannot be correct for both halves of a bimodal population at once, so this case should be, and is, harder for the fitter than the plain exponential one.

Table 4: Under an adversarial mixture (two exponentials with different rates within each cluster, not the fitter's assumed single rate) learned separates significantly from global and tracks oracle closely but not exactly: a gap that is no longer floored at zero, though not significant at $n = 10$.

Lambda source	Median stale-hit-rate	95% CI	Median n_hits	Median n_stale_hits
Global	0.208	(0.170, 0.244)	166	37
Learned	0.080	(0.071, 0.100)	142	11
Oracle	0.074	(0.067, 0.103)	142	11

Learned versus global remains significant, at the same effect size as the well-specified run ($U = 4.0$, $p \approx 0.0005$, $r = 0.92$). Learned versus oracle, though, is now for the first time in this report a comparison with a real, non-degenerate gap to measure: median stale-hit-rate is 0.080 against 0.074, not distinguishable from noise at $n = 10$ ($U = 59.0$, $p \approx 0.50$, $r = 0.18$, a small-to-moderate effect not confirmed significant). Learned is close to oracle but no longer identical to it seed by seed the way it was under a matched or easier-than-matched model, which fits with the mixture being a genuinely harder case for maximum-likelihood exponential fitting.

Hit counts are also higher under the mixture than under the well-specified run, for every lambda source (166/142/142 against 143/130/130.5, Tables 3 and 4): the short-scale half of the mixture, at a fifth of the cluster's mean half-life, makes more entries expire and get regenerated, changing what is resident in the cache at any given time. That is why the two runs' hit rates are not directly comparable in absolute terms, only in the relative ranking across lambda sources within each run.

Taken together, both misspecification attempts point to the same finding: per-cluster fitting beats a single pooled rate by a large, significant margin under an easier-than-assumed model, an exactly-matched model, and a genuinely adversarial one. Whether learned keeps tracking oracle this closely under a still harder mixture, or with less calibration data, is a natural next sweep, but one this report does not run.

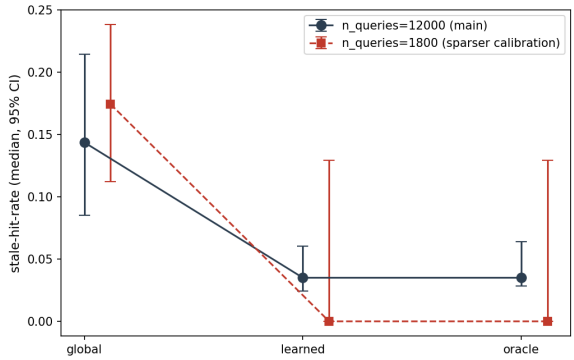

Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the original calibration scale and at a roughly sevenfold sparser calibration scale (both well-specified, i.e. the true half-life is exponential, matching the fitter's assumption). Learned tracks oracle at both scales, as expected under a matched model, and separates significantly from global at both.

5.2 Ablation: isolating the gate and the eviction term

I ran six configurations for 10 seeds each on the same workload, at TTL confidence $c = 0.90$ (this experiment also predates the sweep that sets the shipped default to $c = 0.95$): stock LRU with the gate disabled; stock LFU with the gate disabled, which serves as the floor of comparison given Biton and Friedman's own finding that LRU underperforms on semantic workloads; a documented substitute for Biton and Friedman's released policy, also with the gate disabled; the validity gate combined with plain LFU eviction; the validity gate combined with full FreCoS eviction; and the validity gate combined with FreCoS eviction with the size-normalization term removed.

Table 5: Median stale-hit-rate, hit rate, and useful-hit-rate by row, 10 seeds each. Row 2 (LFU, gate off) is the floor. Useful-hit-rate, the median over seeds of (served and correct and non-stale hits) / (scored queries), is the number that reflects the trade-off: row 2 to row 5 it drops from 0.0170 to 0.0165, about 3%, while stale-hit-rate drops roughly 19-fold.

Row	Configuration	Stale-hit-rate	Hit rate	Useful-hit-rate	Δ stale-hit-rate vs. row 2
1	LRU, gate off	0.661	0.048	0.0170	−0.019
2	LFU, gate off (floor)	0.680	0.052	0.0170	—
3	B&F substitute, gate off	0.680	0.052	0.0170	+0.000
4	LFU, gate on	0.042	0.017	0.0166	−0.638
5	FreCoS, gate on (full stack)	0.035	0.017	0.0165	−0.645
6	FreCoS (no size), gate on	0.042	0.017	0.0165	−0.638

Read as hit rate alone, the gate looks costly: 0.052 down to 0.017, a drop of about a third. Read per query, though, useful-hit-rate tells a different story (reported per seed and then medianed, not derived from the medians of hit-rate and stale-hit-rate, which would give a slightly different number): it only drops from 0.0170 at the floor to 0.0165 at the full stack, about 3%, while stale-hit-rate falls from 0.680 to 0.035, roughly 19-fold. cost_saved_usd stays unchanged across every row (0.44–0.45) for the same reason: it counts only non-stale hits, and the set of queries that hit at all barely moves. The 19x figure in stale-hit-rate is real, but the comparison behind it, against LFU with no freshness mechanism at all, is one any correctly-implemented TTL would win. The more informative comparison sits in Section 5.1: learned against global pooling, a 4x reduction that isolates what *learning* the rate buys on top of a hand-set global TTL.

One row produced an unplanned but informative result: the documented substitute for Biton and Friedman's policy turned out identical to plain LFU in every correctness and cost column,

seed by seed, to full precision. The reason is that at this configuration's roughly 5% hit rate, almost every entry at eviction time has never been accessed a second time, so the substitute's frequency-divided-by-cost score evaluates to zero for essentially every candidate and the policy falls through to LFU's own tie-break. I did not have access to Biton and Friedman's released code in this build environment, and on this workload the substitute is indistinguishable from the LFU floor it was meant to sit alongside, so the eviction axis here is compared only against count-based policies, not the literature's own strongest published policy, which weakens the positioning claim in Section 1.2 that this work compares against the literature rather than the product baseline; that comparison stays qualitative rather than experimental.

The interaction I predicted, that the gate carries most of the improvement while the decay term contributes less on top than it does alone, held in direction, and more sharply than I expected. Isolating the gate's own effect (gate-plus-LFU against the floor) gives $\Delta = -0.638$ ($U = 0.0$, $p \approx 0.0002$, $r = 1.0$); isolating FreCoS's decay term on top of an already-active gate gives a further $\Delta = -0.007$, an order of magnitude smaller and not statistically distinguishable from zero ($U = 50.5$, $p \approx 0.97$, $r = 0.01$). The gate accounts for roughly 99% of the total move from floor to full stack. I checked both components for a shared cluster-identity bug anyway, the specific integration failure I thought most likely; neither disagreed with the other about an entry's cluster.

I expected the decay term to contribute measurably less than it does alone, not nothing at all. The size-normalization row was just as indistinguishable from the full stack on every metric at this cache size. To rule out low eviction pressure as the cause, I reran the comparison at a fifth of this cache size (330 entries, where eviction pressure is far higher), and it held: $U = 50.5$, $p \approx 0.97$, median stale-hit-rate 0.021 with size versus 0.021 without. With the gate already filtering out anything stale, the entries eviction chooses among are uniformly fresh, so neither term differentiates much among them at either cache size.

Table 6: Latency and resource use at the LFU floor (row 2) and the full stack (row 5). Extension overhead and CPU/RSS are measured directly; latency and throughput are dominated by the simulated miss-cost distribution (Section 4.2), not by the extension, and should be read as simulation-dominated, not as a real end-to-end measurement. Latency and throughput rows are a single run, not a median over seeds, and carry no confidence interval, unlike every other table in this report.

Metric	Row 2 (floor)	Row 5 (full stack)
Extension overhead, mean (μs), measured	0.57	0.69
Latency, mean (ms), simulation-dominated	128.9	133.7
Latency, p95 (ms), simulation-dominated	293.1	295.7
Latency, p99 (ms), simulation-dominated	450.3	453.0
Throughput (queries/s), simulation-dominated	7.76	7.48
Peak RSS (MB), median (95% CI)	43.8 (38.7, 44.9)	34.8 (33.7, 39.4)
CPU (%), measured	98.6	99.3

The apparent drop in peak RSS from floor to full stack is not a reliable finding. The two confidence intervals overlap substantially (Table 6), and ten single-process measurements on a shared machine are noisy enough that I would not read anything into this difference without a dedicated, repeated memory-profiling run, which this report does not include. The throughput drop (7.76 to 7.48 queries/s) is not attributable to extension overhead either: it simply follows from the gate converting hits into misses, and a simulated miss costs roughly 100 to 150 times more simulated latency than a real cache-side decision does.

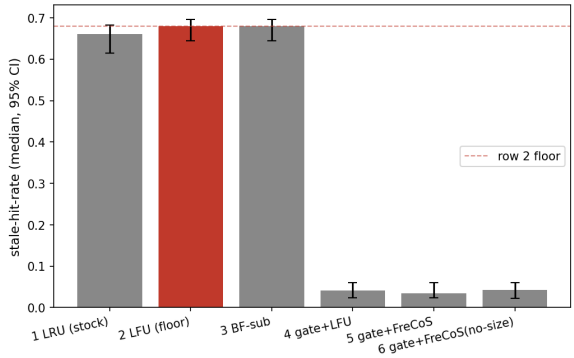

Figure 2: Stale-hit-rate across all six ablation rows, with 95% confidence intervals. Row two, LFU with the gate disabled, is marked as the floor of comparison.

5.3 Cache-size sweep and the correctness-efficiency trade-off

I swept cache size across 5 points spanning 5%–80% of the workload's distinct-answer working set, with the full stack running throughout. Hit rate rises from 0.013 (median 98 hits) at the smallest point to 0.017 (132 hits) at 30%, then flattens out completely; the three largest points produce identical per-seed hit counts. The knee, defined as the point where marginal hit-rate gain per additional entry drops by more than an order of magnitude relative to the previous step, falls at 30% of the working set: the slope drops roughly 13x going into that point and is exactly zero afterward. This cache size anchors every other sweep in this report.

The next sweep produces the trade-off curve this report's argument leans on most heavily: holding cache size fixed at that knee, I sweep the gate's target confidence level.

Table 7: Useful-hit-rate, the number that reflects the trade-off per query, stays flat within noise from 0.80 to 0.95 and only drops at 0.99, even though raw hit rate erodes steadily across the whole range and stale-hit-rate falls 16x from 0.80 to 0.99. Median n_useful_hits (served, correct, non-stale hits) makes the flatness concrete: three matching counts, then a drop at 0.99. Most of the stale-hit-rate gain is captured in the first step, 0.80 to 0.90.

TTL conf.	Med. stale-hit-rate	Med. hit rate	Med. useful-hit-rate	Med. n_hits	Med. n_useful
0.80	0.126	0.019	0.0167	146	126
0.90	0.042	0.017	0.0166	132	125.5
0.95	0.023	0.017	0.0166	129	125.5
0.99	0.008	0.016	0.0162	124	122.5

The exchange is not uniform. Most of the stale-hit-rate benefit comes in the first step, 0.80 to 0.90 (0.126 to 0.042), while raw hit rate erodes at a steadier pace throughout. Read that way, the trade-off looks like it costs something at every step, but it does not. Useful-hit-rate sits at 0.0166–0.0167 from 0.80 through 0.95, backed by matching raw counts (median n_useful_hits 126, 125.5, and 125.5, effectively unmoved), and only drops to 0.0162 (n_useful_hits 122.5) at 0.99. That drop is not confirmed significant at $n = 10$ (Mann-Whitney $U = 33.5$, $p \approx 0.22$, $r = 0.33$), so per Section 4.3's own standard this counts as a directionally consistent, small difference rather than a detected one. Most of what raw hit rate treats as an ongoing cost is really just gate-prevented stale hits being subtracted back out by the same metric that penalized them, not an actual loss of useful service. Any confidence up to 0.95 is close to free in useful-hit-rate terms, and only 0.99 trades a further stale-hit-rate gain for a cost this sample cannot confirm is real.

A third sweep varied cluster count at 5, 10, 20, and 50. This one came out non-monotone on both metrics, with every adjacent confidence interval overlapping heavily, so no directional effect is distinguishable from noise at 10 seeds per point. A plausible mechanism is that more clusters

shrink each one's calibration sample, making the learned rate noisier, while finer-grained clusters could in principle track heterogeneous half-lives more precisely, two effects pulling in opposite directions. This result sits in a supplementary table (`analysis/figures/supplementary.csv`) rather than a headline figure, since it clears neither the effect-size nor confidence-interval bar the rest of this report's figures were held to.

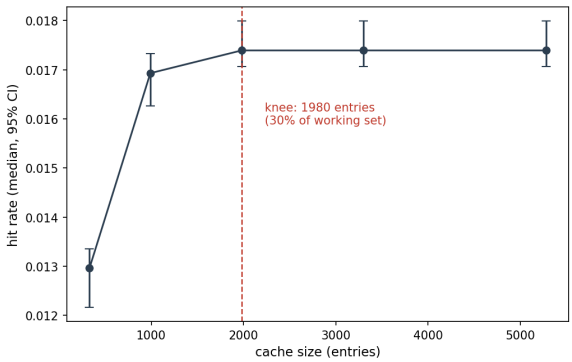

Figure 3: Hit rate against cache size, with the identified knee at 30% of the working set marked.

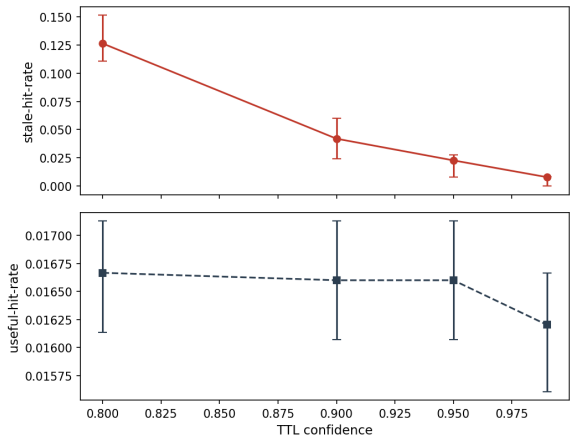

Figure 4: Stale-hit-rate and useful-hit-rate plotted together against the gate's target confidence level. Stale-hit-rate falls 16x over this range; useful-hit-rate stays flat through 0.95 and only drops at 0.99, and that drop is not confirmed significant at $n = 10$.

6 Discussion

Fitting a staleness rate from observed data beats both alternatives I tested, a single pooled rate and no rate at all, by a large, significant margin, under an easier-than-fitted model, a matched model, and a genuinely adversarial mixture model (Section 5.1.1). Under the matched and easier models, learned tracks oracle almost exactly, a narrower result than the "strictly between the two brackets" shape I originally expected: it says per-cluster maximum-likelihood estimation is accurate on this workload, not merely that learning lands partway to a ceiling it approaches but cannot reach. Under the adversarial mixture, a real, non-degenerate gap between learned and oracle finally opens up, though still not one distinguishable from noise at $n = 10$.

I expected the gate to carry most of the stale-hit-rate improvement, with the decay term contributing less on top than it does alone. What I found instead is that the decay term's contribution is not distinguishable from zero at either cache size I tested. The gate is the correctness mechanism here; the eviction term does not add detectable work on top of it, so a

system that only wants the staleness-correctness gain could plausibly adopt the gate alone. That gate is, at bottom, a time-to-live learned per cluster rather than set globally by hand, and this project's strongest result is really a better time-to-live, not some mechanism outside that family: Section 5.1 is exactly that comparison, learned's median stale-hit-rate of 0.035 against global's 0.144, a significant 4x difference, where the global arm stands in for the hand-set time-to-live a system without per-cluster fitting would use instead.

The correctness-efficiency curve in Section 5.3 is less of a trade-off than raw hit rate makes it look. Read on hit rate alone, raising the gate's confidence looks like a steady cost. Read on useful-hit-rate, most of that cost turns out to be stale hits being converted to misses, which the useful-hit-rate metric correctly does not count against the system. Useful-hit-rate sits at 0.0166–0.0167 from confidence 0.80 through 0.95, with matching raw useful-hit counts at those three points, and only drops at 0.99, a drop this report's sample cannot confirm as significant. The practical recommendation is narrower than "pick a point on the curve": any confidence up to 0.95 is close to free, and only 0.99 shows a directionally consistent drop, one this sample cannot confirm.

Three limitations bound how far these results generalize. Every staleness result here comes from a synthetic, self-authored workload. I attempted to get external ground truth from Wikipedia and stopped after a feasibility spike measured 40% rule agreement against an 80% bar set in advance. The harness's exact-match index caps every absolute rate well below what a real semantic index would give, and leaves false-hit-rate carrying no signal (Section 4.2); every finding here is a relative comparison across conditions that share that limitation, and I have not verified that a semantic index would preserve the same rankings. Neither prior-art comparator, Biton and Friedman's released policy nor Cortex's LCFU, was actually run against this workload, so the eviction axis is measured only against count-based policies.

7 Conclusion and future work

I set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and build a system that closes it, with two components matching the two gaps in Section 1.1. The gate did the work I hoped it would: it converts almost all stale hits into misses while removing only about 3% of useful hits, at under a microsecond of overhead per decision, and the learned per-cluster rate that feeds it beats naive pooling significantly across three tested half-life distributions, including a genuinely adversarial one. The eviction term fared worse: its cost and size terms pass every property test, but their measured contribution on top of an active gate is not distinguishable from zero at either cache size tested. One gap is closed and measured; the other's mechanism is implemented and tested but has not been shown to add value here.

A real semantic index in place of the exact-match harness would likely change every absolute number here and is the change most likely to make results production-comparable; I expect the relative findings to survive it, but have not verified that. Running Biton and Friedman's policy directly would close the comparator gap in Section 6. Calibrating the generator's distributions against a real inference trace would strengthen external validity, once the Azure trace or an equivalent becomes reachable. Dynamic re-clustering, a FreshCache-style probabilistic staleness budget, and admission control remain future work.

Appendix A: correctness evidence

Correctness is checked at three levels. A reference oracle (`tests/oracle/reference_cache.py`) implements the same hit/miss/stale decision as the extension, but using a naive dict-and-linear-scan approach with no shared code, and every pipeline decision is cross-checked against it in `tests/test_stock_parity.py`. A callable invariant suite (`tests/invariants.py`) runs not only in unit tests but also inside `benchmarks/harness.py` on every experimental run, checking

budget bounds, argmin eviction selection, that no entry is served past its cluster's time-to-live, and that staleness decisions derive from creation time, never last-access time. Property-based tests in `tests/test_frecos.py` check that the eviction value function is monotonic in each of its four terms independently.

The most persuasive of these artifacts is a property test that failed against my original design rather than a bug I found by inspection: `test_cold_start_entry_scores_above_zero` in `tests/test_frecos.py` asserts that a fresh entry must score above zero, and it failed against the literal log(1+freq) formula. That failure is how I found and fixed the log(1+freq) → log(2+freq) issue described in Section 3.1.

Appendix B: code and data artifacts

This repository is public at https://github.com/LiorLotan2/frecos. A `Dockerfile` and `environment.yml` build a working environment from a clean clone, and `.github/workflows/ci.yml` reruns the full test suite and a benchmark smoke test on every commit. `benchmarks/samples/smoke_row.csv` and `smoke_row.json` are committed sample benchmark logs reproducible with `make bench-smoke`.

Table 8: Where every artifact referenced in this report lives in the repository.

Artifact	Location
Pipeline seam (serve-path decision)	`gptcache_ext/pipeline.py`
Additive metadata storage	`gptcache_ext/metadata.py`
Staleness model (clustering, fitter, gate)	`gptcache_ext/staleness/`
Eviction (FreCoS and baselines)	`gptcache_ext/eviction/`
Synthetic workload generator (W1)	`workloads/w1_synthetic/`
Benchmark harness and metrics	`benchmarks/harness.py`, `metrics.py`
Experiment runners (Section 5)	`benchmarks/runners/`
Raw results per experiment	`results/brackets/`, `ablation/`, `sweeps/`
Figure regeneration (one command)	`cd analysis && python3 make_figures.py`
Reference oracle	`tests/oracle/reference_cache.py`
Invariant suite	`tests/invariants.py`
Property tests	`tests/test_frecos.py`
GPTCache pinned commit and source-map	`docs/baseline-source-map.md`
Wikipedia feasibility spike	`docs/w2-feasibility.md`
Docker / environment for a clean rebuild	`Dockerfile`, `environment.yml`
Continuous integration	`.github/workflows/ci.yml`
Sample benchmark logs	`benchmarks/samples/`

No figure in this report is hand-edited; every one is regenerated directly from a committed results CSV by the single command in Table 8.

References

[1] N. Biton and R. Friedman. *From Exact Hits to Close Enough: Eviction Policies for Semantic Caches*. arXiv:2603.03301, 2026.

[2] C. Ruan, C. Bi, K. Zheng, Z. Shi, X. Wan, and J. Li. *Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching*. arXiv:2509.17360, 2026. NSDI 2026; earlier preprint circulated under the title "Asteria."

[3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.

[4] C. Wang, X. Liu, Y. Zhu, A. Youssef, P. Nagpurkar, and H. Chen. *Category-Aware Semantic Caching for Heterogeneous LLM Workloads*. arXiv:2510.26835, 2025.

[5] M. Mansoor, T. Ahmad, and Y.-C. Yoon. *Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs*. arXiv:2607.04281, 2026.

[6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.

[7] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.

[8] G. Einziger and R. Friedman. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.

[9] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.

[10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. https://github.com/zilliztech/GPTCache, 2024. Version 0.1.44, commit bae7ffeef774e762d9d4e60fce70be00011188a6.

[11] P. Rajpurkar, R. Jia, and P. Liang. *Know What You Don't Know: Unanswerable Questions for SQuAD*. arXiv:1806.03822, 2018.